

Why these 5 were selected			
Baseline	Main capability being tested	Important parameters/capabilities	Why we selected it
1. DeepFT	Proactive edge fault tolerance	Fault prediction, fault diagnosis, self-supervised learning, task scheduling, migration, QoS, overhead	Closest overall competitor to your complete system
2. PreGAN	Proactive fault prediction + recovery	Anomaly detection, fault diagnosis, migration prediction, QoS, recovery/overhead	Tests whether your causal-adaptive approach improves over proactive AI-based FT
3. IPFT	Resource prediction + proactive FT	CPU/RAM/disk/bandwidth prediction, RNN, HBES adaptation, replication, migration, reliability, maintainability	Tests your method against prediction-driven adaptive fault tolerance
4. BWOAIF	Real-time adaptive anomaly detection	Streaming detection, anomaly score, online adaptation, concept drift, computation/memory	Tests your real-time + adaptive detection claim
5. RCD	Causal root-cause localization	Causal graph, failure intervention, root-cause metric/service localization	Tests your causal diagnosis/localization contribution directly

1. DeepFT — closest overall baseline

What it does

DeepFT uses a **deep surrogate model** to predict and diagnose faults, combined with co-simulation-based self-supervised learning so the model can adapt to changing conditions. It then uses the predictions to make **proactive scheduling and migration decisions**.

The paper explicitly focuses on:

- fault prediction
- fault diagnosis
- self-supervised learning
- adaptation
- task scheduling
- task migration
- QoS

It reports improvements in fault-detection and QoS, including reductions in deadline violations and response time. [one_DeepFT.pdf](#)

Parameters/capabilities that matter for us

Detection

- Fault score

- Fault prediction
- Fault diagnosis

Proactive FT

- Scheduling decision
- Migration decision

Performance

- F1
- QoS/SLO violations
- Response time
- Model overhead/scalability

Why we choose it

This is the **most important baseline** because it is already trying to solve something very close to your complete problem.

Your argument becomes:

DeepFT provides proactive AI-based fault tolerance, while our approach additionally uses causal reasoning to identify the responsible cause and adapt recovery according to the causal system state.

So if your method beats DeepFT on MTTD, false alarms, localization and recovery, that is meaningful.

2. PreGAN — proactive prediction + migration

What it represents

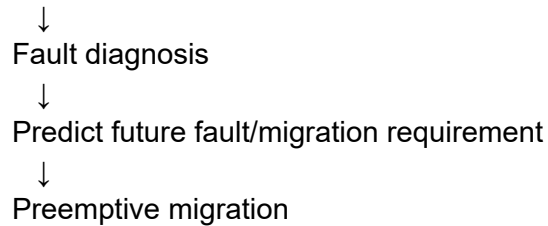
PreGAN represents another **AI-based proactive fault-tolerance approach**.

The important idea is:

Telemetry



Anomaly/fault detection



This is valuable because your system is also supposed to **act before the service becomes seriously degraded**.

Parameters we care about

- Fault detection accuracy
- Fault diagnosis accuracy
- Prediction capability
- Migration decision
- QoS impact
- Recovery/migration time
- Computational overhead

Why we choose it

DeepFT and PreGAN are similar in the broad goal but use different AI mechanisms.

That gives you a stronger experiment:

"Our method is not merely better than one particular deep-learning approach; it performs better against multiple proactive AI-based fault-tolerance strategies."

Important distinction

We should **not use the original PreGAN numerical results as if they were TON-IoT results**.

We will adapt/reimplement the core approach for your common experimental environment and measure it there.

3. IPFT — prediction-based adaptive fault tolerance

This one is **very important for your project**.

IPFT monitors resource usage such as:

- CPU
- RAM
- bandwidth
- disk

and predicts future resource usage using RNN-based models.

It then uses those predictions to trigger:

- node replication
- task migration

and uses **Hybrid Bayesian Evolution Strategy (HBES)** for automated adaptation of the resource prediction models.

Parameters we care about

Prediction

- RMSE
- MAE

Resource state

- CPU
- RAM
- bandwidth
- disk

Adaptation

- HBES
- changing workloads
- heterogeneous edge nodes

Fault tolerance

- replication
- task migration
- reliability
- maintainability

The paper specifically says its proactive mechanism predicts resource problems and triggers migration/replication before the problem occurs.

two_Intelligent_proactive_fault_tolerance_at_the_edge.pdfPDF

Why we choose it

Your proposed method claims to be **adaptive and proactive**.

IPFT gives you a very strong comparison:

IPFT:

Resource prediction



Expected overload



Migration / replication

YOUR:

Telemetry



Fault detection



Causal diagnosis



Severity / future state



Adaptive recovery

So you can demonstrate that **prediction alone is not necessarily enough**; identifying the causal source can improve the recovery decision.

4. BWOAIF — real-time adaptive detection baseline

This is the baseline that directly tests your **real-time + adaptive** claim.

BWOAIF is an online version of Isolation Forest designed for streaming data and concept drift. It continuously receives data, classifies it as normal/anomalous, and updates the forest by removing old trees and creating new ones.

The paper emphasizes:

- streaming data
- concept drift
- online adaptation
- low computational requirements
- few hyperparameters
- scalability

three_bilateral.pdfPDF

Parameters we care about

Detection

- Anomaly score
- Normal/anomaly classification
- MTTD
- False Alarm Rate

Adaptation

- Concept drift response
- Gradual adaptation
- Sudden-change adaptation

Edge suitability

- CPU time
- Memory
- processing overhead

Why we choose it

Suppose your algorithm says:

"I am better because I work in real time and adapt to changing workloads."

The professor can ask:

"But BWOAIF is already real-time and adaptive. What's new?"

Excellent question.

Therefore BWOAIF is an **essential challenge baseline**.

Your experiment needs to demonstrate that your method doesn't merely adapt—it **adapts while using causal information to make better fault-tolerance decisions**.

5. RCD — causal root-cause baseline

This one is different from the other four.

RCD directly addresses the question:

"What actually caused the failure?"

The paper models a system using multiple metrics over time and focuses on localizing the failure despite propagation effects.

One of its important motivations is that failure propagation makes root-cause identification difficult because a failed service can affect other services/metrics.

The paper's approach does not rely on predefined domain knowledge or simple parametric assumptions for its causal graph.

four_root-cause-analysis-of-failures-in-microservices-through-causal-discovery.pdfPDF

Parameters we care about

The key one is:

Causal Localization Accuracy

And additionally:

- localization latency
- root-cause ranking
- number of candidate causes
- computational overhead

Why we choose it

This is the baseline that directly challenges your **causal component**.

Without RCD, someone could criticize:

"You compared your causal system only with anomaly detectors. Of course yours looks better at causal localization."

RCD prevents that criticism.

You can say:

"We compared our causal diagnosis mechanism against an established causal root-cause approach, rather than only against conventional anomaly detectors."

That is much stronger.

The important part: what each baseline tests

This is probably the **best table to show your professor**:

Your claimed capability	Baseline that challenges it
Real-time fault detection	BWOAIF
Adaptive fault detection	BWOAIF
Proactive fault prediction	DeepFT, PreGAN, IPFT
Fault diagnosis	DeepFT, PreGAN
Causal root-cause localization	RCD
Adaptive resource management	IPFT
Proactive recovery	DeepFT, PreGAN, IPFT
Task migration	DeepFT, PreGAN, IPFT
Fault tolerance	DeepFT, PreGAN, IPFT
Low edge overhead	BWOAIF, DeepFT, IPFT
QoS preservation	DeepFT, PreGAN, IPFT

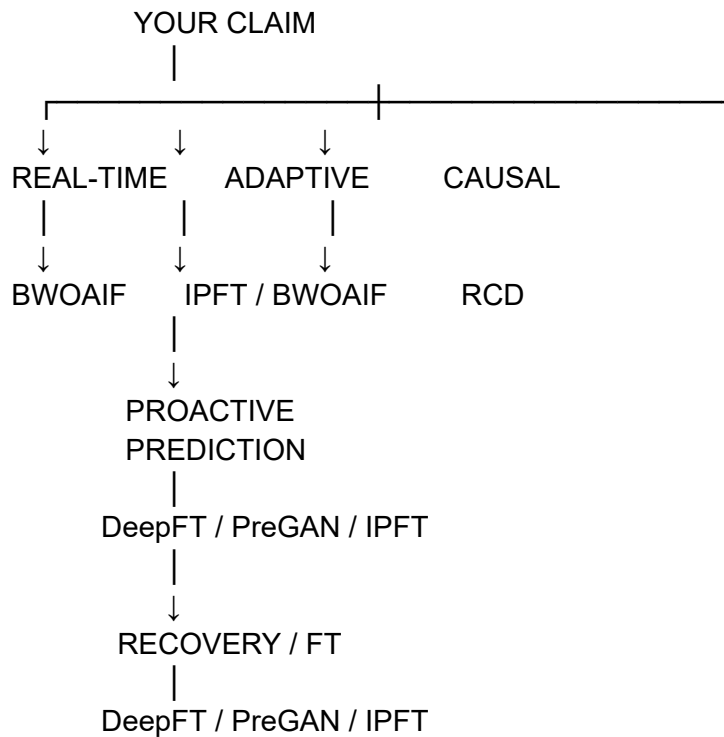
So the five are **not redundant**.

And this is exactly why I prefer these 5

Your project claims something like:

Causal + Real-Time + Adaptive + Fault-Tolerant

We can map it directly:



Therefore the comparison is meaningful:

BWOAIF asks:

"Can your method detect/adapt better than an online adaptive detector?"

DeepFT asks:

"Can your method improve on proactive AI-based edge fault tolerance?"

PreGAN asks:

"Can your method improve proactive fault prediction and migration?"

IPFT asks:

"Can your method improve prediction-based adaptive resource fault tolerance?"

RCD asks:

"Can your causal mechanism actually localize causes better than an existing causal RCA method?"

And finally:

Your method asks:

"Can I combine all of these capabilities into one causal, real-time, adaptive fault-tolerance loop?"

The seven metrics then make sense

Metric	Why we measure it
MTTD ↓	How quickly the method detects an emerging fault
Early Warning Time ↑	How early it identifies a fault before service failure
False Alarm Rate ↓	Whether adaptation/causal reasoning avoids unnecessary alarms
Causal Localization Accuracy ↑	Whether it identifies the actual responsible cause
Recovery Success Rate ↑	Whether the chosen action actually restores the service
End-to-End Recovery Time ↓	How quickly the system returns to normal
Edge Computational Overhead ↓	Whether the approach is practical on constrained edge nodes

One caution

For **RCD**, recovery metrics aren't naturally part of the original method. For **BWOAIF**, recovery isn't its primary function either.

So we should **not artificially give them zero recovery success**.

Instead, when we implement the final experiment, we'll decide on a **common recovery layer** that can take each method's detection/diagnosis output and make a recovery decision under the same conditions. That will allow a fair end-to-end comparison.

In short: these five were chosen because together they are the strongest challenges to each major claim of your proposed system—not because they all perform the same task.